
AMD EPYC Processor Performance Playbook

Preview

Feb 13, 2025

CONTENTS:

1	Getting Started	3
1.1	How to Use This Document	3
1.2	Glossary of Terms	5
1.3	Complementary Tuning Documents	5
1.4	Referenced Tools	6
2	System Checks	9
2.1	Access Level	9
2.2	CPU Properties	9
2.3	System Software Checks	11
2.4	Performance Variability Checks	12
2.5	Checking and Setting CPU Affinity	13
3	Performance Analysis Motivators	15
3.1	Changes Across System Stacks	15
3.2	EPYC Cross-Generational Performance	15
3.3	Different Behaviors Relative to Non-EPYC Systems	15
3.4	Performance Variability	16
3.5	Latency and Tail Latency	16
3.6	Application Performance Scaling	16
3.7	Even Better Performance and/or Power	16
4	Performance Indicators	17
4.1	Collecting High-Quality Performance Indicators	17
4.2	High-Level Software Indicators	18
4.3	High-Level Hardware Indicators	20
4.4	CPU Performance Indicators	22
5	Common Application-Aware Tuning Opportunities	25
6	EPYC-Specific Behaviors and Opportunities	27
6.1	Optimizing L3 Domain Usage	27
7	CPU Pipeline-Based Opportunities	37
8	Application Scaling Opportunities	39
8.1	Types of Performance Scaling	39
8.2	Scaling Efficiency	39
9	Capturing Indicators Using AMD uProfPcm	41

10 Processor-Specific Solutions	43
10.1 Adaptive Allocation	43
10.2 Speculative Stores Mode Configuration	43
11 External References	45
12 Additional Resources and Legal Notices	47
12.1 Revision History	47
12.2 Notices	47
12.3 Trademarks	47

Publication Number: 63994 Revision: 1.0

Date: December 2024

Restricted — AMD Internal Distribution Only

GETTING STARTED

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

1.1 How to Use This Document

Strong performance analysis and debug practices often consist of establishing a reproducible run-time environment, setting performance expectations, and then identifying where the underlying assumptions and expectations are diverging from reality.

This AMD EPYC Processor Performance Playbook works through the above process systematically, with many specific and common examples for accomplishing each step. By following these steps, many performance optimization opportunities will be discovered. The goal is, in the best case, for all performance questions to be resolved, and in the worst case, the performance engineer is pointed in a better direction than when they started.

It is not required to read the document cover-to-cover, or to attempt every described system tweak. The approach is to collect a focused set of indicators and to use that information along with the user's understanding of their application to find focused, intuitive optimization opportunities.

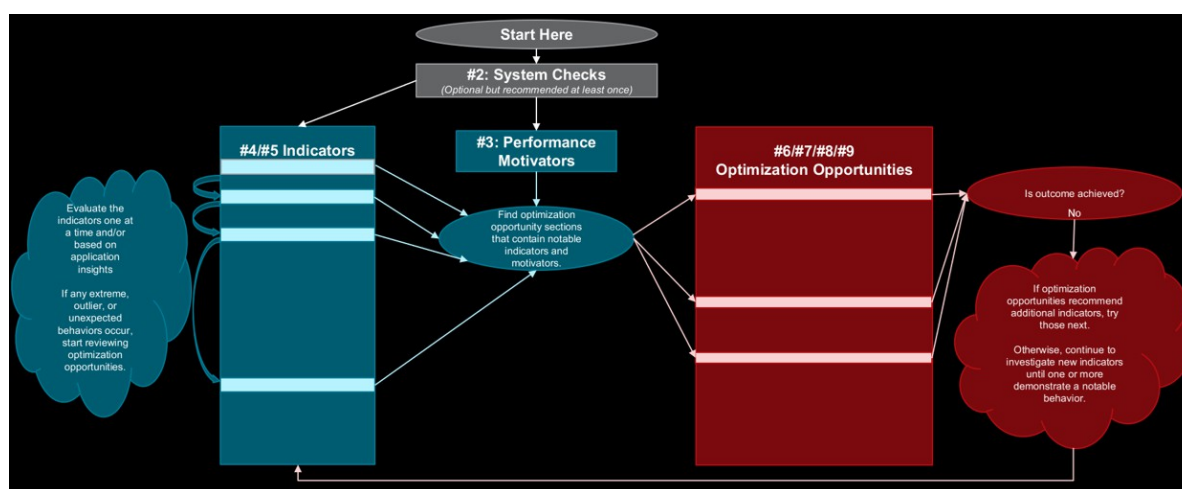


Fig. 1: Figure: Playbook Performance Optimization Flow

The performance optimization methodology underlying this document is based on the following main steps (illustrated in *Figure: Playbook Performance Optimization Flow*):

- *System Checks*: Generate a performance understanding of the system under debug through a basic set of system checks. This includes confirming a variety of critical software and hardware attributes.
- *Performance Analysis Motivators*: Categorize the performance observation that is motivating the performance optimization. These are referred to as motivators in this document.
- *Performance Indicators*: Collect a key set of indicators spanning system hardware, software, and processor microarchitecture that help indicate which aspects of the system are most likely driving, constraining, or impacting the delivered or expected performance. This section can be traversed in whatever order makes the most sense to the user. The section provides guidance on what can be considered extreme behaviors for these indicators, and user can begin attempting focused optimizations. In this document indicators can serve as a “thread” of performance optimization. For example, searching the document for `Low Per-CPU Utilization %` will reveal how to measure it, it’s likely causes and associated trade-offs, possible solutions that help optimize it, and areas for further investigation.
- Use the *System Checks*, *Performance Analysis Motivators* and *Performance Indicators* to find the most common optimization opportunities for the application under analysis. The focused optimizations cover the following:
 - *Common Application-Aware Tuning Opportunities*: These include hardware and software optimizations that take advantage of an application’s characteristics, and are applicable to x86 server performance in general.
 - *EPYC-Specific Behaviors and Opportunities*: This section covers hardware and software optimizations that specifically take advantage of features of AMD EPYC processors.
 - *CPU Pipeline-Based Opportunities*: This section covers optimization opportunities gleaned from microarchitectural indicators, which may lead to general or EPYC-specific opportunities.
 - *Application Scaling Opportunities*: This section covers performance optimization opportunities to enable proper exploitation of the high throughput capabilities offered by AMD EPYC processors.

In addition, this document provides the following resources:

- *Getting Started*: Glossary, Documents, and EPYC architecture basics
- *Processor-Specific Solutions*: EPYC Generation-Specific Guidance
- Tool-specific guidance on how to collect performance indicators mentioned in this document (*Capturing Indicators Using AMD uProfPcm*)

In each optimization opportunity section or sub-section, there is a table that references the high-level indicators and motivators that could or should have led to that section. It has the following format.

High-Level Indicators

- Indicator 1
- Indicator 2
- ...

Possible Motivators

- Motivator 1
- Motivator 2
- ...

The goals of mapping these insights into optimization suggestions are two-fold: 1) to help provide insight into the relationships between observations and the available tuning options, and 2) reduce the number and types of optimizations that end up being considered and/or applied.

The optimization opportunity sections then go on to offer two types of actions:

- **Possible Solutions**: These are specific actions to apply and then re-evaluate performance. The action will be described specifically in the document or a link to specific directions in another available document will be

provided. Possible solutions will typically involve altering an established system parameter, such as a BIOS value, an OS parameter, compiler option, or source code changes.

- **Further Investigation:** These are directional suggestions to continue the debugging process. The Playbook will have guided the optimization process to a more focused area and provided resources that may assist with these further investigations.

The intent is for the strategies and guidance in this document to apply to 3rd Gen, 4th Gen, and 5th Gen EPYC. If and when generation-specific directions are required, that will be stated explicitly.

1.2 Glossary of Terms

Term	Definition
Physical Cores	The basic compute engine of an AMD EPYC Series Processor. For example, an AMD Genoa CPU may have up to a maximum of 96 physical cores). A physical core may appear to software as one or two logical CPUs, depending on whether Simultaneous Multi-threading (SMT) is enabled.
Hardware Threads	A hardware thread is the same as a physical core if SMT is disabled. When SMT is enabled the number of available hardware threads on the CPU is 2x the number of physical cores.
Logical CPUs	Numerically this is the same as the number of hardware threads available in the system and is more commonly used when discussing system resources from the perspective of software. So each physical core can be viewed as having one hardware thread or logical CPU (when SMT is off) or two hardware threads or logical CPUs when SMT is on. For example, on an SMT enabled, single-socket top-of-the stack (96 core) AMD Genoa system, the OS will see 192 logical CPUs. If running inside a virtual machine on a single-socket AMD Genoa host that has SMT enabled, depending on how the Virtual Machine is set up, the guest operating system may see anywhere from 1 to 192 logical CPUs. Alternatively, if SMT is not enabled on the host, the virtual machine could see anywhere from 1 to 96 logical CPUs.
NUMA Node	When different parts of main memory have non-uniform access latency, such as when main memory is distributed over two sockets, each region of main memory and its associated logical CPUs are referred to as a NUMA node.
L3 Domain	An L3 domain includes all logical CPUs that interact directly with the same shared L3 cache. Typically, a single socket has one or more L3 domains. For EPYC, L3 domain is synonymous with a Core Complex (CCX).
Cache Hierarchy	Cache hierarchy refers to the multiple levels of cache within an L3 domain. For example, L1 instruction cache and op cache, L1 data cache, L2 cache, and L3 cache. It does not include any interactions beyond the L3 domain.

1.3 Complementary Tuning Documents

1.3.1 BIOS and Workload Tuning Guides

Default BIOS options generally produce the best overall performance for generic workloads, but these defaults may not be optimal for a specific workload. AMD continually tests various workloads; these tuning guides discusses BIOS options to deliver both maximum performance and performance-per-watt.

Processor Family	AMD EPYC Series	Link
Turin	EPYC 9005	BIOS and Workload Tuning Guide for AMD EPYC 9005 Series Processors
Genoa	EPYC 9004	BIOS and Workload Tuning Guide for AMD EPYC 9004 Series Processors
Milan	EPYC 7003	Workload Tuning Guide for AMD EPYC 7003 Series Processors
Siena	EPYC 8004	BIOS & Workload Tuning Guide for AMD EPYC 8004 Series Processors

1.3.2 Architecture Programmer's Manual (APM)

This volume is intended for programmers writing application programs, compilers, or assemblers. This volume describes resources and functions of the AMD64 architecture that are accessible to application software, including memory, registers, instructions, operands, I/O facilities, and application-software aspects of control transfers (including interrupts and exceptions) and performance optimization.

[AMD64 Architecture Programmer's Manual, Volumes 1-5](#)

1.4 Referenced Tools

1.4.1 Linux Perf Utility

“perf” is a profiling tool for Linux based systems that abstracts away CPU hardware differences in Linux performance measurements and presents a simple command line interface. Perf is based on the `perf_events` interface exported by recent versions of the Linux kernel.

The minimum upstream Linux kernel version needed to try commands in this document is Linux v6.2+. Since distributions tend to backport upstream patches piecemeal to the level the distribution ships at, unless the distribution kernel provided is v6.2+, your mileage may vary with the commands, even if the distribution claims they have backported the patches. There are plenty of online resources to explore for linux perf usage (See:[Perf Wiki](#))

1.4.2 Linux hwloc

“lstopo” based commands are reference frequently in this document. Lstopo is available as part of hwloc, which is a valuable utility for examining the cores, cache, memory, and I/O architecture of any CPU- based system, but may need to be installed. It is part of the **Portable Hardware Locality** library, and a package is available on all popular Linux distributions (See: [Linux hwloc man pages](#))

1.4.3 AMD uProf

AMD uProf is a software profiling analysis tool for x86 applications running on Windows, Linux and FreeBSD operating systems and provides event information unique to the AMD “Zen”-based processors. AMD uProf enables the developer to better understand the limiters of application performance and evaluate improvements (See: [AMD uProf](#)).

1.4.4 Flame Graph

Flame Graphs are useful visualizations of code optimization opportunities. There are many online resources describing Flame Graphs (See: [Flame Graphs](#) and their [Usage](#)) Standard profiling tools like [AMD uProf](#) and [Linux Perf Utility](#) can be used to generate Flame Graphs.

1.4.5 numactl

NUMA is a computer architecture design that separates memory into multiple domains that are directly attached to a subset of processors. System architectures with NUMA tend to have better scalability, but efficient use of memory requires a conscious effort to affinity scheduling and allocating memory in the correct domain. Linux offers a command-line utility called numactl that provides a user control of

NUMA (Non-Uniform Memory Access) policies, process – CPU affinities, and process-memory affinities. Developers and system administrators use numactl to achieve this better affiliation. (See: [numactl](#)).

Numactl enables processes or tasks to be restricted to specific CPU and memory areas and decreases traffic in the system memory. This results in a reduction in cache misses, performance degradation, and latency. Additionally, with the usage of numactl, more efficient use of CPU, memory, and I/O devices can be obtained, and help to resolve memory sharing issues.

1.4.6 bpftrace

eBPF (enhanced Berkeley Packet Filter) (See: [eBPF](#)) enables a user to extend the functionality of the kernel without actually making changes to it. bpftrace (See: [bpftrace](#)) is a high-level tracing language for eBPF available in recent Linux kernels (4.x onwards) and can help provide detailed information about kernel or hypervisor behaviors to help drive performance optimization.

SYSTEM CHECKS

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

2.1 Access Level

At the outset, it is important to be aware of the access level the user has over system resources, as this may influence which specific performance analysis techniques are available. In decreasing order of access level:

Complete access: On a bare-metal system this implies root access, or the ability to run specific commands with root-level privileges (example: `sudo` in Linux). In a host running one or more virtual machines this implies root access in the Guest Operating System running on the Virtual Machine, as well as root access to the host system and hypervisor

Limited access: This could imply various levels of access.

1. User level access in a guest operating system with no access to the host
2. Root level access in a guest operating system with no access to the host
3. User level access in a bare-metal system

This document provides numerous HOWTOs for collecting certain performance indicators, detecting underlying system configurations, and applying tweaks to change those configurations. Depending on your system administrator, the access level you need for executing these commands may vary.

If running with **Limited Access**, it may be necessary to inspect and adjust `kernel.perf_event_paranoid`, working with your administrator.

2.2 CPU Properties

It is important to be aware of the various attributes of the CPU listed in this section.

2.2.1 CPU Model and Number

To start with, run the following command to obtain the CPU Model Number. This is particularly important if running on a virtual machine. Check with the host system administrator to ensure that the virtualized CPU enables the latest ISA features offered by the host CPU. Otherwise, application software will not be able to use all the performance

features of the underlying hardware. The same command also provides the number of logical CPUs available on the system. This is referred to as `Max Available CPUs` in this document.

Tools	<code>lscpu</code>
Commands	<code>lscpu grep "Model name:"</code>

2.2.2 SMT

Use the following command to determine the number of cores, threads (logical CPUs), and sockets in the system. On a bare metal system, the number of threads (or logical CPUs) per core will be either 1 or 2, depending on whether SMT is enabled in the BIOS. On a virtual machine, this may be 1 or 2, depending on the CPU topology exposed to the guest by the hypervisor. Use the below to determine if SMT is enabled on the system. In most cases, SMT will be beneficial for performance.

Tools	<code>lscpu</code>
Commands	<code>% lscpu grep -E '^Thread ^Core ^Socket ^CPU\ ('</code>
Attributes	SMT enabled / SMT disabled

2.2.3 Sockets

Determine the number of sockets on the system. Multi-socket systems can significantly increase throughput, while cross-socket activity can influence performance.

Tools	<code>lstopo</code>
Commands	<code>% lstopo-no-graphics grep Package</code>
Key Attributes	Number of Sockets

2.2.4 NUMA Domains Per Socket

Determine the number of NUMA nodes per Socket (NPS) and associated CPUs and memory. Even in a single socket system, AMD EPYC™ Series Processors support multiple NUMA domains per CPU each of which have high performance. However, cross-NUMA domain activity can influence performance of the application.

Tools	<code>numactl, lstopo</code>
Commands	<code>% numactl --hardware</code> <code>% lstopo-no-graphics</code>
Key Attributes	The above <code>numactl</code> command displays on stdout for each NUMA node, a list of CPU numbers and memory capacity associated with each NUMA node. Total Available NUMA Nodes can be measured using <code>% numactl --hardware grep size wc -l</code> <code>lstopo-no-graphics</code> prints system topology information to stdout. From this infer: $\text{Nodes Per Socket (NPS)} = \text{Total available NUMA nodes} / \text{Number of Sockets}$

The optimum choice of how many NUMA domains per socket to use is highly depends on application characteristics and the specific EPYC generation under consideration.

2.2.5 L3 Domains

Determine the number of L3 domains & logical CPUs per domain. AMD EPYC processors support the notion of an L3 domain which enables very high CPU counts and excellent TCO benefits. For performance optimization, it is important to have L3 topology information at hand because cross-domain interactions (analogous to NUMA) can have an impact on application performance.

Tools	lstopo, lscpu
Commands	% lstopo-no-graphics grep L3 wc -l Shows the number of L3 cache domains in the system.
	% lscpu grep "^CPU(s) :" Shows the Max Available CPUs in the system
	% lstopo-no-graphics Lists a view of CPUs, Threads, L1 and L2 Caches under each L3 domain, NUMA domain and socket domain.

Determine the amount of L3 cache per socket and per L3 domain. The amount of L3 cache per domain and per socket can vary depending on the specific AMD EPYC™ Series Processor. It is important to be aware of these properties because of how they influence application performance.

Using lscpu

Tools	lscpu
Commands	% lscpu grep "L3 cache:" Shows the size of one L3 cache.
	% sudo lshw -C memory grep -A 4 'L3 cache' Shows the total L3 size on each node

Using lstopo

Note: This might require the `hwloc` package to be installed.

Tools	lstopo
Commands	% lstopo (might be available as <code>lstopo</code> , <code>lstopo-no-graphics</code> , or <code>hwloc-ls</code>)
Key Attributes	Shows the machine topology and hierarchy.

Using sysfs

Tools	sysfs
Commands	% cat /sys/devices/system/cpu/cpu*/cache/index3/ shared_cpu_list sort uniq
Key Attributes	Each line corresponds to a unique L3 domain and shows the CPUs associated with the said L3 domain.

2.3 System Software Checks

Here is a system software checklist to merit review before detailed performance analysis. These are fundamental configurations and settings that may not always be enabled by default but can have meaningful impact on performance. This document generally assumes that these have already been checked and optimized.

2.3.1 Kernel Version

Determine the version of the linux kernel on the system, irrespective of whether the operating system is running on a virtual machine or bare metal.

Tools	uname
Commands	% uname -a

Check how far ahead the upstream kernel is (see: [*The Linux Kernel Archives*](#)), what major performance features interim kernels may have. If upgrading the kernel is not an option, check if your distribution has back-ported performance related patches.

2.4 Performance Variability Checks

An important check to perform before embarking on a detailed performance optimization initiative, is to start from a repeatable baseline level of performance. Performance debug and optimization typically involves iterating on hypotheses and making “A vs. B” comparisons. It’s important that the phenomena being evaluated and tuned are relatively constant.

After determining the system configuration and ensuring the system is set up to be performant as described in the prior sections, run a repeated set of performance measurements of the application to assess how stable the reported scores are, and what type of performance variability, if any, they present. Repeatability across reboots and other identically built systems is also expected.

Examples of the types of performance variability that applications may exhibit include:

- **Bimodal variability:** some runs produce a set of high scores, others a set of low scores.
- **Outliers:** most runs have the same score, but occasional runs show much better and/or worse.
- **Monotonic:** consistent gain or loss of performance over runs.
- **Random:** scores vary to a large degree with no discernible pattern.
- **Jitter / Tail Latencies:** a small number of iterations/operations have much higher latency than others.

Below are common factors known to introduce variability in measured performance.

- **CPU-External Factors** (notably networking and storage): If non-CPU components may be introducing the observed variability, try to eliminate these factors by either normalizing the state of the devices across runs or by tuning the application to reduce sensitivity to the device’s state.
- **CPU Affinity:** Limiting the application to a fixed number of CPUs can help reduce run to run variability. Even if this is not possible in a production environment, it is useful whenever conducting accurate performance optimization experiments. For an application running in a guest OS, there are two levels of binding. One is which set of guest CPUs the application is constrained to. The next level is which set of host logical CPUs the virtual machine is mapped to. Use [Checking and Setting CPU Affinity](#) (See: [*Performance Variability Checks*](#)) for information on setting CPU affinities in both bare metal and virtualized environments.
- **Background Processes:** Reduce / eliminate all background processes that may interfere with the performance of the application under study.
- **Noisy Neighbors in Virtualized Environments:** If the application is running on a guest operating system sharing physical machine resources with other virtual machines, there may be variations in performance. For controlled performance experiments, run your VM in isolation.
- **Database or File Size Bloat:** Check if there are left over effects from prior runs. For example, if benchmarking a database workload, the database may grow in size over time and reduce the performance of subsequent runs.

2.5 Checking and Setting CPU Affinity

For performance analysis, debug, and optimization, in many parts of this document recommend setting affinities to control where in the system the processes run, and where memory is allocated. This leads to much higher controllability of system performance to understand the performance context of an application. This is done by constraining the application to use a limited set of logical CPUs and portions of physical memory so that downstream measures of performance indicators and what-if analysis can be limited to them. This may also be a valid technique to improve performance and performance predictability in production environments (example: HPC workloads). Also consult on-line resources on this topic (see: [Affinity](#)).

How to check CPU affinity for application processes

Tools	numactl, taskset, ps
Commands	% grep cpus_allowed /proc/<pid>/status % taskset -c -p <PID>
	To check where the PID is actually running, check under CPUID: % ps -o pid,cpuid,comm -p <PID>

How to set CPU affinity

Tools	numactl, taskset
Commands	Linux numactl provides numerous options to control CPUs and Memory being used by a workload: <pre>% numactl -C <list of cpus to limit the workload to> --<process command></pre> <pre>% numactl -C <list of cpus to limit the workload to> --localalloc --<process command></pre> <pre>% numactl -C <list of cpus to limit the workload to> --M<NUMA node numbers> --<process command></pre> Using flavours of numactl, one can limit the workload to an L3 domain, a few L3 domains, a specific NUMA node, a socket, etc. <pre>% taskset --cpu-list <list of cpus> <process command></pre> or, if the process is already running: <pre>% taskset -pc <list of cpus> <pid></pre>

PERFORMANCE ANALYSIS MOTIVATORS

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

Many observations and situations can motivate the analysis, debug, and optimization of system performance. This section discusses a few common motivators. The purpose is to help focus the optimization process. The performance motivators defined here may be referenced in the later sections that focus on specific system optimization steps. Look for the motivators below under the “Possible Motivator” header in the optimization subsections.

3.1 Changes Across System Stacks

Motivator: Changes Across System Stacks

It is common to observe a change in performance for the same processor when the system stack changes. Some examples include updates or changes to the OS, firmware BIOS, using different I/O devices, and introducing virtualization. Performance will often remain neutral or improve as the stack is changing, but it may also regress. The sources and reason behind a performance regression can be difficult to isolate, as the hardware and software interactions can change extensively. This guide will cover some of these reasons.

3.2 EPYC Cross-Generational Performance

Motivator: Cross-Generation Performance Delta

When comparing performance across EPYC processor generations, the expectation is that performance goes up with the newer processor, all other things being equal. However, as the architecture evolves over time, the interactions with the system software or other hardware components may subtly change and result in a performance regression. These regressions are typically recoverable. This guide will cover some of the common scenarios that can lead to this, and how to resolve them.

3.3 Different Behaviors Relative to Non-EPYC Systems

Motivator: Non-EPYC Behavior Differences

There are many reasons to compare application behaviors and performance across different systems or processors. The differences observed while making comparative assessments will, in general, not be uniform across applications. This is because some applications may be sensitized differently by the EPYC design relative to non-EPYC systems.

While CPUs from different vendors have many similarities (including Instruction Set Architecture), each processor architecture is the result of many different tradeoffs. Some of these tradeoffs are visible to software development and if not properly understood, can be performance visible as well. If the software stack is heavily tuned and optimized on one architecture, it is unlikely to be optimal for another. This document will point out unique software-visible aspects of EPYC that differ from other available processors and corresponding strategies to optimize for them.

3.4 Performance Variability

Motivator: Performance Variability

Occasionally, applications may exhibit performance variability that cannot be easily removed and/or explained by the factors discussed earlier [See [Performance Variability Checks](#)]. Resolving this variability is outside the scope of the document. However, it is still possible to continue the analysis. In such cases, automatically collect the high-level performance indicators [See [Performance Indicators](#)] over a series of runs. Identify a “good” run and a “bad” run and follow the performance analysis guidance in the following sections to isolate what is causing the lower performance in the “bad” run.

3.5 Latency and Tail Latency

Motivator: Latency Targets

Many applications are required to meet specific latency targets. For example, an application may require that average response time seen by a user or client should be less than a specific threshold latency, or that 99% of such requests should be satisfied under some other threshold (commonly referred to as an SLA, or “Service Level Agreement”). Meeting such latency targets while maximizing throughput requires an awareness of various high-level indicators, and possible optimization opportunities. Where applicable, this document will point these out.

3.6 Application Performance Scaling

Motivator: Unexpected Application Scaling

As the number of hardware threads per processor increases, the number of software threads must grow as well to efficiently utilize all the capabilities of the system. The system can scale out by adding more instances of applications, or the applications can scale up by adding more software threads. The rate at which application performance scales relative to the increase in number of application software threads is often referred to as the application scaling. During the process, disproportionated performance improvements may be attributed to scaling capability of the application. There is a targeted section on [Application Scaling Opportunities](#) goes into this background in greater detail and nuance along with highlighting the optimization opportunities most likely to improve scaling efficiency.

3.7 Even Better Performance and/or Power

The most common reason to optimize is simply to improve the efficiency of the systems, i.e. faster responses, higher performance throughput, larger degree of determinism, less power, etc. In this case, the entire range of optimization opportunities are potential candidates. So it is best to collect as many of the indicators as possible [See [Performance Indicators](#)] and focus on optimizations associated with outlier values of those indicators.

PERFORMANCE INDICATORS

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

This section describes a set of performance indicators to collect at run-time with instructions on how to collect them using various tools. The first two sections, *High-Level Software Indicators* and *High-Level Hardware Indicators* respectively assess if basic software behaviors and utilization of hardware resources are sane. Each indicator is qualified by threshold levels that provide indication of whether performance optimization opportunities exist. If so, there are pointers to other sections of this document that describe the opportunities in detail and how to exploit them.

This is followed by a section on *CPU Performance Indicators* which cover indicators that measure first-order and detailed bottlenecks in the CPU pipeline.

Before performing any measurements, review *System Checks* to ensure that the system is set up for performance. In particular, determine the *Access Level*, since this will impact which measurements can and can't be performed. Example: in a guest operating system in a cloud environment, the set of available indicators will greatly depend on the cloud service provider.

Next, ensure the application is invoked in a representative manner with realistic inputs, data sets or problem sizes. Third, scores and performance indicators (whether they be software or hardware), should be measured during the “steady state” of the application. That is, they should not be corrupted by behaviors peculiar to application start up and tear down phases. The goal should be to collect indicators in regions of code execution that will result in real-world performance improvement.

4.1 Collecting High-Quality Performance Indicators

Several of the tools referenced present a view of the run-time characteristics of the software that is executing on the CPU, or how such software is exercising the underlying hardware. For all such tools, it is critical to ensure that the characterization data or performance indicators are collected accurately. For this:

- Ensure that data collection is started and stopped in the intended phase or *steady state of the application* run, avoiding any system idle time, start up, or tear down phases.
- Limit data collection and analysis only to logical CPUs where the application is actively running. Many performance indicators for a logical CPU are reported in PTI terms (per thousand instructions) and could be misleading if it is running at low utilization, as the denominator (instruction count) can be low. It is advisable to examine the indicator and utilization at a per-CPU level first, rather than directly consuming the average.
- *Sampling*: Most tools will report the number of samples collected during data collection. Some tools also allow for setting the sampling frequency. Too low a frequency will lead to poor confidence in the reported indicators due to insufficient coverage of performance events. Too high a frequency could lead to high performance

overhead. Usually, the tool defaults should work fine, but it is useful to keep an eye on the total number of samples collected, and the overhead of data collection on the performance of the application.

4.2 High-Level Software Indicators

This section describes a basic set of indicators about software behavior that can provide hints about performance opportunities.

4.2.1 Code Profiling

The starting point for software indicators is typically a `Code Profile`. A `Code Profile` provides a view of which code is exercised most often across the application, libraries and operating system. This profile typically highlights where the application is spending most of its time, and hence should guide where optimization effort should be focused. Often, this step alone provides enough clues to optimize an application.

It should be kept in mind however, that many modern server applications have dynamic behavior. In general, the paths taken by code is not guaranteed to be identical across runs. There are various reasons this could happen, such as asynchronous events, thread scheduling, work-stealing worker threads, timing-dependent code paths, polling, synchronization etc. These software behaviors may be influenced by hardware performance.

This means that the same application may exhibit different instruction level behaviors across different systems and processors.

For most of the *Performance Analysis Motivators* described earlier it is useful to check if software is behaving (functionally speaking) in the same way across two experiments. This is especially important given many of the hardware indicators described in this document are expressed in terms of a hardware event (example: cache miss rates or branch mispredicts per thousand instructions (PTI)) Knowing if and when PTI indicators are comparable is important. Hence being able to collect and evaluate a `Code Profile` based on “instructions” in addition to “cycles” is important.

How to create a code profile

Note: To ensure that the `Code Profile` of the kernel is properly represented, kernel debug symbols need to be installed. Most Linux distributions do not have debug symbols by default, but they can be downloaded and installed. Additionally, to properly reconstruct call graphs for the user space, the necessary compiler options must be enabled to preserve call-stack information for the application.

Tools	perf
Commands	There are many ways to use perf to generate a <code>Code Profile</code> , for which extensive online documentation exists. For example, the following command generates a profile of the relative contribution of different routines to the total time spent. If this is run without root privileges it will profile only user activity. With root privileges, it will profile both user and kernel code: <pre>% perf record -F 99 -a -e cpu-cycles -- sleep <time to profile, in seconds> % perf report --stdio</pre>
	It may be useful to separately profile user and kernel behaviors <pre>% perf record -F 99 -a -e cpu-cycles:u -e cpu-cycles:k -- sleep <time to profile, in seconds> % perf report --stdio</pre>
	Instead of <code>-e cpu-cycles</code> , using <code>-e instructions</code> , the above will generate a profile of the relative contribution of different routines to the total number of instructions executed.

Later parts of this section discuss specific indicators that can be extracted from a `Code Profile`. Indicators are `% time` or `% instructions` spent in specific processes and functions.

4.2.2 Synchronization Overheads

A common source of performance overhead in server applications is software synchronization. It is useful to pay specific attention to and quantify these overheads explicitly. Check if the `Code Profile` reveals high overheads arising from synchronization related routines. This could be user space synchronization (example: `pthread_mutexes`, OpenMP barriers) and/or kernel space synchronization (example: `spinlocks`, `queued spinlocks`). Note, some software synchronization primitives force waiting threads to consume CPU cycles, and in scenarios where they contribute to the overhead, they will naturally appear in the `Code Profile` (example: `spinlocks`). Other types of primitives could be blocking in nature, and force the waiting thread to relinquish the CPU, freeing the CPU up to do other work. In such cases, the percentage contribution of such routines by themselves (such as a blocking `pthread_mutex`) to a `Code Profile` may not provide an accurate view of the overhead.

How to measure synchronization overheads

Tools	perf
Commands	From the <code>Code Profile</code> collected above, accumulate the <code>% time</code> spent in routines that are synchronization related. Examples provided below. • <code>pthread_mutex_lock</code> • <code>gomp_barrier_wait_end</code> • <code>gomp_team_barrier_wait_end</code> • <code>futex_wake</code> • <code>futex_cleanup</code> • <code>futex_wait</code> • <code>native_queued_spin_lock_slowpath</code> • <code>pv_queued_spin_lock_slowpath</code> • <code>_raw_qspin_lock</code> • <code>_raw_qspin_lock_irq</code> • <code>_raw_spin_lock</code> • <code>_raw_spin_lock_bh</code> • <code>_raw_spin_lock_irq</code> • <code>_raw_spin_lock_irqsave</code> • <code>_raw_spin_trylock</code> • <code>_raw_spin_unlock_bh</code> • <code>_raw_spin_unlock_irqrestore</code> • <code>rcu_read_lock</code>
Key Indicators and Thresholds	Synchronization Overhead: Accumulate <code>% time</code> spent in above routines Low Synchronization Overhead: < 10% Medium Synchronization Overhead: 10-20% High Synchronization Overhead: > 20%

Next Steps

If Medium Synchronization Overhead or High Synchronization Overhead is detected, investigate techniques to reduce these overheads by taking advantage of software visible attributes of the EPYC architecture.

- [Optimizing L3 Domain Usage](#)

4.3 High-Level Hardware Indicators

This section describes a set of high-level performance indicators that capture how the application is using various hardware resources at a high level, and how to collect them. These indicators provide first level indication of what is limiting the performance the application, and an initial characterization of how the hardware is responding to its demands. These indicators can be used as guidance for next level performance analysis or optimization. Unless otherwise indicated, hardware indicators are expressed in PTI units (Per Thousand Instructions).

4.3.1 Logical CPU Usage and Utilization

The number of active logical CPUs (see [Glossary of Terms](#) for what this refers to) and their utilization levels are important performance indicators. This distinction between the percentage of active CPUs (CPU Usage %) and how active each of those CPUs are (Per-CPU Utilization %) is important, and these terms can be easily overloaded or conflated.

A particular logical CPU is either halted or executing instructions. The percentage of time that it is executing instructions (i.e. not halted) will be referred to as Per-CPU Utilization %. If this utilization is below 2%, a logical CPU can be considered inactive and otherwise it is active. The percentage of logical CPUs that are active will be referred to as CPU Usage %. The absolute CPU Usage is simply CPU Usage % x Max Available CPUs.

Note that High Per-CPU Utilization % by itself does not always mean useful work is being done (i.e. software threads may be busy waiting on a spinlock or a barrier). Utilization is an incomplete indicator on its own but is useful when combined with other data.

How to measure Per-CPU Utilization %

Tools	htop or AMD uProf
Commands	Use any CPU or system load monitoring tool (such as htop) to generate Per-CPU Utilization % for all the CPUs in the system. Alternatively, see Capturing Indicators Using AMD uProfPcm
Key Indicators and Thresholds	- Inactive Per-CPU Utilization %: <2% (does not count towards CPU Usage %) - Low Per-CPU Utilization %: 2-30% - Moderate Per-CPU Utilization %: 30-90% - High Per-CPU Utilization %: >90%

How to measure CPU Usage %

Tools	htop or AMD uProf
Commands	Use any CPU or system load monitoring tool to generate Per-CPU Utilization % for all the CPUs in the system. Alternatively, see Capturing Indicators Using AMD uProfPcm
Key Indicators and Thresholds	- CPU Usage = Count the number of CPUs for which CPU Utilization% > 2%. - CPU Usage % = (CPU Usage / Max Available CPUs) x 100 - Low CPU Usage %: 0-30% of Max Available CPUs are active - Moderate CPU Usage %: 30-90% of Max Available CPUs are active - High CPU Usage %: >90% of total Max Available CPUs are active

Next Steps

Any headroom in CPU usage % or Per-CPU Utilization % represents compute resources that are not being used efficiently. Note that, while maximizing these indicators may help improve throughput-oriented application-level performance metrics, it may come at the cost of *Latency and Tail Latency* metrics. Care must be exercised in determining and driving towards an appropriate level of CPU Usage % and Per-CPU Utilization % for the application under consideration while experimenting with techniques for optimizing CPU usage described in *Common Application-Aware Tuning Opportunities*.

4.3.2 Instructions Per Cycle (Without Halt)

IPC w/o halt is the number of instructions retired per cycle by a logical CPU while it is active, i.e. it is not in an idle state. This is a fundamental indicator of its performance. In fact, Per-CPU-Utilization, CPU Frequency Mhz, and IPC w/o halt when multiplied out provide a measure of the achieved Per-CPU Throughput. Multiplying this by the number of active logical CPUs (CPU Usage) provides a measure of the Total Instruction Throughput seen by the application.

Note that this is true for the most part, when instructions represent useful work performed in an application context. There can be exceptions where threads may be “busy-waiting” a resource (See *Synchronization Overheads*). In such cases, an improvement in Total Instruction Throughput may not translate to improved application-level performance metrics.

The absolute value of IPC w/o halt by itself is often less valuable as a performance indicator compared with relative values, in the context of a certain change to the system. For example, for many of the suggested performance optimizations in this document, the first order impact will be seen on IPC w/ o halt.

How to measure IPC w/o halt

Consult *Capturing Indicators Using AMD uProfPcm* for instructions.

4.3.3 L3 Cache Domain Usage

A distinguishing property of many AMD EPYC™ Series Processors is the presence of multiple L3 cache domains per NUMA node. An L3 domain consists of multiple physical cores (and hence logical CPUs) connected to a shared last-level cache. The L3 domain is an important architectural concept for performance optimization in AMD EPYC™ Series Processors. Hence, as a first step, determine the number of L3 domains in your system and the CPUs sharing each domain using commands described in *L3 Domains*.

Frequent cross-L3 domain data sharing can lead to performance inefficiencies, depending on the type of accesses. The indicator below captures the percentage of L1 data cache misses that result in cross-domain data sharing. Simply having a high percentage of cross-domain data sharing is not necessarily a performance issue. This needs to be assessed along with other indicators (such as the absolute rate of DC Fills) to determine if there may be a performance optimization opportunity.

How to determine CPU cross-domain activity levels

Tools	AMDuProfPcm
Commands	Consult <i>Capturing Indicators Using AMD uProfPcm</i> for instructions.
Key Indicators and Thresholds	CPU Cross-L3 Domain Data Sharing % Low CPU Cross-L3 Domain Data Sharing %: <2% (less than 1 out of 50 L1 cache misses) Moderate CPU Cross-L3 Domain Data Sharing %: 2-5% (1 out of 50 to 1 out of 20 L1 cache misses) High CPU Cross-L3 Domain Data Sharing %: >5% (more than 1 out of 20 L1 cache misses)

Next Steps

If Moderate CPU Cross-L3 Domain Data Sharing % or High CPU Cross-L3 Domain Data Sharing % is detected, performance optimization can be high impact, especially for software that may have been developed for non-EPYC systems. For guidance targeting efficient use of the L3 domains in AMD EPYC™ Series Processors, consult *Optimizing L3 Domain Usage*.

4.4 CPU Performance Indicators

4.4.1 Pipeline Utilization

Starting with Zen4 based CPUs, tools like *Linux Perf Utility* and *AMD uProf* can generate a set of special indicators that provide a “Pipeline Utilization” view of the microarchitecture of the Zen Core pipeline.

This view provides, at a high level, what microarchitectural bottlenecks in the core pipeline each logical CPU is experiencing.

On every clock cycle, the Zen core can retire a certain number of “operations”, the exact number depending on the “width” of the core (example: Zen4 can dispatch and retire 6 ops simultaneously). The total number of pipeline slots is this width, multiplied by number of clock cycles in a measurement window. Pipeline utilization shows us the fraction of slots used to retire the operations, and the fraction that was un-utilized due to various types of stalls, attributed to common microarchitectural factors.

How to generate Pipeline Utilization Indicators

Tools	AMDuProfPcm
Commands	Consult <i>Capturing Indicators Using AMD uProfPcm</i>
Key Indicators and Thresholds	<i>Expected Ranges of Pipeline Utilization for Various Application Domains</i> <i>Pipeline Utilization Indicator Threshold Ranges</i>

Interpreting “Pipeline Utilization” Indicators

Next, expected ranges are provided for these indicators, as well as information on what to do if the results for the application fall outside those ranges.

The *table below* provides the expected ranges for the Pipeline Utilization indicators across a few application domains on AMD 4th Generation EPYC (code-named “Genoa”) processors. Note that these should be treated as high-level guidance and can be loosely applied to other EPYC generations. This provides some sense of whether the application may have outlier behaviors.

Table 1: Expected Ranges of Pipeline Utilization for Various Application Domains

Pipeline Utilization	ML/HPC	Enterprise	INTEGER	JAVA
% frontend_bound	5-10%	30-40%	15-20%	10-20%
% bad_speculation_mispredicts	<5%	<5%	5-10%	<5%
% smt_contention	5-15%	10-15%	15-20%	10-20%
% backend_bound	60-70%	30-40%	25-30%	45-55%
% backend_bound_memory	40-65%	30-40%	15-30%	35-45%
% backend_bound_cpu	15-30%	5-15%	5-15%	10-20%
% retiring	15-20%	10-15%	20-25%	10-20%

The *table below* provides indicator thresholds to guide optimization opportunities.

Table 2: Pipeline Utilization Indicator Threshold Ranges

Pipeline Utilization	Low	Moderate	High
% frontend_bound	< 10%	10-20%	> 20%
% bad_speculation_mispredicts	< 5%	5-10%	> 10%
% smt_contention	< 10%	10-20%	> 20%
% backend_bound	< 25%	25-50%	> 50%
% backend_bound_memory	< 20%	20-50%	> 50%
% backend_bound_cpu	< 5%	5-20%	> 20%
% retiring	< 10%	10-25%	> 25%

High % frontend_bound indicates the application is bottlenecked by the beginning, or front, of the CPU's pipeline (responsible for code fetches, instruction decode, branch prediction, and page translations for code accesses) did not supply instructions fast enough to keep the later stages, or back, of the pipeline (responsible for instruction execution, memory hierarchy accesses, and retirement) busy.

High % bad_speculation indicates that a substantial number of pipeline slots were wasted by operations that are speculatively dispatched but never retired (i.e. owing to the mispredicted branches).

High % smt_contention indicates that a substantial number of pipeline slots were lost to an inability to select an SMT thread that had instructions ready for dispatch.

High % backend_bound indicates that a large fraction of slots were lost to stalls introduced by the later stages, or back, of the pipeline. Determine if the primary contributor to High % backend_bound is High % backend_bound_memory or High % backend_bound_cpu.

High % backend_bound_memory or Moderate % backend_bound_memory indicates that the bottlenecks are related to long latency memory access operations. For solutions consult the following sections of the playbook:

- Optimizing Main Memory Bandwidth
- Optimizing Main Memory Access Latency
- *Optimizing L3 Domain Usage*
- Tuning Data Cache Prefetchers
- Tuning Cache Replacement Strategies
- Cross-Socket Traffic

High % backend_bound_cpu indicates resource contention in the execution units in the back of the CPU pipeline.

% retiring: This refers to slots that are usefully utilized to retire instructions and is closely correlated with instruction / cycle or IPC. The higher this value, the better. As performance is optimized through the techniques suggested in this document, this indicator should improve.

Note: This is a preview of the Playbook. Later versions will contain actionable steps based on the above indicators.

COMMON APPLICATION-AWARE TUNING OPPORTUNITIES

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

The default configuration of EPYC processors are designed to provide great performance for all applications. However, there are commonly utilized options that can potentially deliver better performance for a specific application. This chapter will discuss such opportunities.

EPYC-SPECIFIC BEHAVIORS AND OPPORTUNITIES

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

Software optimizations developed with or targeted toward other processor architectures typically translate well to EPYC. However, there are additional scenarios where AMD EPYC can deliver differentiated performance thanks to novel architecture tradeoffs. Therefore, there are common scenarios where modifying software specifically for the architecture of AMD EPYC™ Series Processors may provide noticeable efficiency increases.

6.1 Optimizing L3 Domain Usage

High-Level Indicators

- High % backend_bound_memory OR Moderate % backend_bound_memory
- High CPU Cross-L3 Domain Data Sharing % OR Moderate CPU Cross-L3 Domain Data Sharing %
- High L3 Cache Miss OR Moderate L3 Cache Miss
- High Per-CPU Utilization % OR Moderate Per-CPU Utilization %
- High Synchronization Overhead
- High System Time % OR Moderate System Time %

Possible Motivators

- Performance Variability
- Unexpected Application Scaling
- Non-EPYC Behavior Differences
- Cross-Generation Performance Delta

Before going further, note that the trade-offs and optimizations that will be discussed for interactions across L3 domains are analogous to cross-socket data sharing for traditional multi-socket processors. Therefore, much of this discussion involves applying the same mindset and techniques to a different processor organization layer.

Note that, in this section, “domain” will refer to an L3 domain and “cross-domain” will refer to interactions across those L3 domains.

L3 domain performance optimization opportunities will be based on the amount of fine-grain data address sharing that occurs between different L3 domains. If the data for a shared data address requested by one L3 domain is resident in

the cache hierarchy (L1, L2, or L3 cache) of a different L3 domain, then the latency to access the data is significantly higher – on the order of a DRAM main memory request – than a cache hierarchy hit in the same L3 domain. The peak cross-domain bandwidth is also lower than the bandwidth to the local cache hierarchy.

Infrequent cross-domain data sharing is not typically a performance constraint, and the latency is mitigated by standard processor techniques, like out-of-order execution and data prefetching. However, frequent cross-domain activity can become a first-order performance limiter and is preventable.

An important note is that “clean” data continues to be cacheable across multiple caches – and therefore multiple L3 domains – concurrently. Clean data means that the cacheline (aligned at a 64B granularity) is effectively read-only. Dirty data sharing refers to cachelines that are expected to be written, even occasionally. In general, clean data sharing is less likely to impact performance than dirty data sharing.

The rest of this section discusses different conditions under which data addresses can be shared and the optimization opportunities for each scenario.

Possible Solutions

The first solutions set of solutions may be helpful regardless of the underlying reason for the cross-domain sharing. Solutions related to specific sources of cross-domain interactions are described later in this section.

- **Modify the Speculative Store Mode:** The Speculative Store Modes change how aggressively/ speculatively the processor tries to write to a cacheline. More aggressive can help when there is less mis-speculation and/or less contention. Less aggressive can help when there is more mis-speculation and/or high contention.

BIOS options exist starting on Zen4 and may change across processor generations. This type of guidance is typically available in generation-specific BIOS tuning guides [See *BIOS and Workload Tuning Guides*]. In this instance, instructions on how to manipulate this feature are provided in the processor-specific section of this document [See *Speculative Stores Mode Configuration*].

6.1.1 Shared Data Structures

Sharing data structures across multiple software threads is a common practice. For example, multiple threads may access the entries of a shared buffer. Under most circumstances, these shared structures are not accessed often enough to require any special performance optimization with multiple L3 domains.

However, there are scenarios where these data structures are shared and updated frequently (example: debug structures, logging structures, microbenchmarks/stress tests).

Identify the code and or main memory locations that are sharing data using any of the methods described below.

How to attribute L3 domain interactions to specific code and data

Tools	perf
Commands	<pre>% perf c2c record -- <cmd></pre> Or <pre>% perf record -e ls_any_fills_from_sys.near_cache -c 1000 % perf report -i perf.data</pre> Analyze the annotation to determine which specific operations and data structures are responsible for the L3 domain interactions. If the above does not yield enough information, the following is a less direct, but often useful method for the same purpose. Run the application in two ways: • Use a low number of logical CPUs (say 8) to contain threads inside one L3 domain. • Rerun using two L3 domains but using half the available Logical CPUs/domain (say 2x4). Note, the second config can provide some performance benefits (<i>CPU Scheduling to L3 Domains: Spreading vs. Packing</i>) but if the application has a large number of updates to shared data, performance degradation from spreading will be measurable, and attributable to specific code and data as follows: <pre>% perf record -C <list of CPUs> -F 99 -e cycles % perf record -C <list of CPUs> -F 99 -e instructions % perf report #after each instance of perf record</pre> Post process the data collected as follows: • Identify the hottest functions (using the “cycles” profile) • Calculate IPC for these functions (using “instructions” profile) • Compare IPC of these functions in the spread v packed config • Determine functions with the largest IPC loss in the spread config. Any function seeing an IPC drop of > 5% may be of interest. Run the following on the spread config for further analysis: <pre>% perf record -C <list of CPUs> -F 99 -e ibs_op// % perf report</pre> From this profile, find the functions with the most significant IPC drop identified previously. Use this profile to find the hottest instructions in those functions. They will yield the code and address of the shared data structure responsible for cross- domain activity.

Occasionally, the offending functions may be in the kernel. Use call stack tracing or collect a *Flame Graph* to determine the most frequent call sites that lead to the kernel function responsible for the cross-domain interactions

Tools	bpftrace
Commands	<pre>% bpftrace -e 'kprobe:<kernel_function> {[kstack] = count(); }'</pre>
Secondary Indicators	Examine the call stack to find the user code functions that are calling the kernel function with low IPC w/o halt.

If the above analysis reveals a heavily accessed shared data structure, explore techniques described below to mitigate the behavior.

Further Investigation

- Create multiple copies of the structure to reduce number of software threads that are frequently sharing data. Merge/combine information across the structures at a coarser granularity.
- Eliminate writes to the shared structure. A read-only shared structure is cheaper to cache. Similarly, refactor code to avoid different threads accessing different fields in a structure (unless all accesses are reads).

- Limit sharing of data structures to L3 domain granularity.
- User thread-local or CPU-local data structures and algorithms whenever possible (example: preferentially use the `ThreadLocalRandom` class in Java which does not depend on globally shared data for generating random numbers, unlike the `Random Class`).

6.1.2 Inter-Thread Synchronization

The indicators and motivators listed at the beginning of this section apply (See *Optimizing L3 Domain Usage*), especially `Moderate Synchronization Overhead` or `High Synchronization Overhead`.

Synchronization between software threads involves some form of agreement to decide on a future course of action. This can be implemented using a variety of primitives, based on the programming language, programming model, expected behavior of the synchronizing threads, etc. At the most basic level inter-thread synchronization involves one or more cache lines that are read (potentially many times). Once it reads an expected value a thread will then write a value to the same location. Since multiple threads will access the same locations in main memory (which are cacheable), inter-thread synchronization can result in interactions across L3 and even NUMA domains.

Typically, optimized code will have limited synchronization, and it is unlikely to be a performance limiting factor. However, as synchronization accesses increase in frequency and the number of software threads participating in the synchronization increase, it's more likely to impact performance. This is true regardless of L3 domains, but it can become more exaggerated if multiple L3 domains are involved.

Even when there is no explicit synchronization at the application level, accesses to kernel-controlled resources (example: files, network interfaces) and data structures can result in synchronization overheads.

Evidence of this will be the presence of synchronization primitives such as spinlock, mutex, futex, barriers, etc. in the *Code Profile*.

How to determine the source of the synchronization overheads and critical sections

Tools	Linux perf
Commands	Use Linux perf record and report commands as described in <i>Code Profiling</i> . For routines that indicate synchronization (spinlock, queued spinlock, barrier, mutex, futex, etc) use call graph sampling to detect the most frequent call sites that lead to the synchronization hotspot. <i>Flame Graphs</i> may be useful for this. Browse the code to examine what functionality / data structure is being protected by the most frequent call site leading to a hot synchronization primitive.

Tools	AMD uProf
Commands	<code>AMDuProfCLI collect --config threading -g -o <output-dir> <app></code>
	For more information, refer to <i>Threading Analysis</i> section of the uProf User Guide. To collect the callstack use <code>-goption</code> . Identify when threads are waiting on pthread calls or wait system calls (with timeline). This will provide a summary of wait times, sleep times and runs of a thread. The Wait object summary section of the report helps identify the objects responsible for causing threads to wait.

Possible Solutions

- Ensure the kernel has the latest upstream or has back ported patches (*Kernel Version*) for optimized thread scheduling in EPYC systems
- Affinitize or co-schedule threads that are synchronizing with each other to the same L3 domain (*Checking and Setting CPU Affinity*).

- For OpenMP applications, evaluate policies for thread placement and barrier behavior (OMP_PLACES, OMP_SCHEDULE, OMP_PROC_BIND, OMP_WAIT_POLICY) [See: [openmp](#)]
- Consider running multiple smaller instances of the application and use scale-out methodologies (*Types of Performance Scaling*) rather than a single large instance, to reduce likelihood of cross domain interactions between synchronizing threads.
- If synchronization is being enforced using spinlocks (i.e. *High Spinlock %* is detected), a few specific optimization opportunities exist.
 - Identify the source of the spinlocks using *Code Profiling* (with call graph tracing enabled) or a *Flame Graph* and examine if spinlocks can be avoided (example: by using user space blocking primitives like pthread_mutex)
 - Check that “Test-and-Test-And-Set” discipline is followed in the implementation of the spinlock (see example below) and PAUSE instructions are used (illustrated below using the __mm_pause() intrinsic)
 - Implement techniques to relinquish the CPU on long waits (example: sched_yield()).
 - Consider using the MONITORX/MWAITX instructions on the waiting thread to potentially free up power for other threads

```

/* LOCKED = 1; UNLOCKED = 0 */
// acquire lock

while (1) {

    while (*lock != UNLOCKED)
        __mm_pause();

    if (__sync_bool_compare_and_swap(lock, UNLOCKED, LOCKED))
        break;

}

```

Further Investigation

Numerous well-defined practices exist to optimize the performance of parallel software [*Art of Multiprocessor Programming*]. AMD EPYC™ Series Processors with their industry leading core counts and modular design make it all the more crucial that parallel applications with synchronizing threads be developed with scalability in mind.

Consider standard practices for architecting high-performance concurrent software. For example, ensure the use of fine-grained locks (implying that the resources being protected are at the smallest granularity) to reduce contention. Performance-optimize lock-guarded code to ensure that locks are held for as little time as possible [*Art of Multiprocessor Programming*].

Consider reducing the number of threads synchronizing with each other by using scale-out principles (*Types of Performance Scaling*) to application design.

Choose the right synchronization primitive that is most appropriate for the application and the architecture. For example, Test-and-Set locks have no fairness guarantees and can result in starvation of some threads at high contention levels. Consider using pthread_mutex locks, which guarantee fairness, over spinlocks. Some fair locks, such as qspinlock, maintain fairness by serving requests in the order of arrival. However, they may perform poorly in environments where the lock holder/waiter can be de-scheduled (example: on an oversubscribed system and, on virtualized environments). To obtain the fairness advantage of queued spinlocks for Linux guests under KVM enable pvspinlock.

How to enable paravirt spinlocks in a guest

Tools	libvirt
Commands	Add the following in libvirt domain xml. <pre><features> <pvspinlock state='on' /> </features></pre>

6.1.3 Producer and Consumer

The indicators and motivators listed at the beginning of this section apply (*Optimizing L3 Domain Usage*).

Producer/consumer is a planned sharing relationship between two software threads, which typically involves both threads accessing the same main memory locations via a software-allocated buffer.

One thread will write the locations (producer) and then the other thread will later read the locations (consumer). For peak performance, these components should take place between threads in the same L3 domain. This allows cached data to be shared at low latency.

However, producer/consumer relationships can be difficult to predict and therefore difficult to co-schedule to an L3 domain. In other cases, the relationship may not just be one-to-one. Certain algorithms will rely on, say, 1-to-N relationships, where one producer thread shares with up to N consumer threads. If the ‘consumer’ component ends up on the performance critical path, then performance can be impacted if the participating threads span across multiple L3 domains.

There are alternate modes in the hardware that can be beneficial to critical cross-domain producer/consumer scenarios.

Possible Solutions

- Modify the Adaptive Allocation Caching Mode: The CPU caches may dynamically change policies depending on observed application behaviors. The goal is to increase cache hit rates. However, sometimes it may decrease cache hit rates or an increase in cache hits is not better for an application. The latter can happen in producer/consumer scenarios.

BIOS options exist starting on Zen5 and may change across processor generations. This type of guidance is typically available in generation-specific BIOS tuning guides [See *BIOS and Workload Tuning Guides*]. In this instance, instructions on how to manipulate this feature are provided in the processor-specific section of this document [See *Adaptive Allocation*].

Further Investigation

Otherwise, if cross-domain producer/consumer scenarios have become performance critical for an application, software may need to be refactored to continue to push performance.

- Co-schedule the producer and consumer threads in the same L3 domain. [See: *Checking and Setting CPU Affinity* and *L3 Domains*]
- Change when or how the consumer thread is executed so that the latency of reading the producer’s buffer is overlapped with other work. For example, explore software techniques such as pipelining and prefetching (*The Art of Multiprocessor Programming*).
- Limit the number of consumers per producer, i.e the ‘N’ in 1-to-N producer/consumer. For example, explore hierarchical sharing options where smaller groupings of thread communicate on a fine-grain basis. There are numerous academic and non-academic publications in this area (*The Art of Multiprocessor Programming*, *Chen et al*, *Vlassov et al*).
- The producer can proactively push data that will be imminently consumed, out to memory through non-temporal writes (MOVNTI) or the CLFLUSH instruction. [See: Vol 1 of the *Architecture Programmer’s Manual (APM)*].

6.1.4 Thread Scheduling Effects

The indicators and motivators listed at the beginning of this section apply (*Optimizing L3 Domain Usage*).

Thread scheduling choices can influence the amount of data sharing amongst L3 domains.

Thread migration refers to the operating system scheduler activity of pausing a thread on one CPU and resuming it on a different CPU. One performance implication is that code and data cached on the original CPU may still reside there and could be accessed from the new CPU. Even though it is the same thread, this will cause data sharing between two cores and potentially between two L3 domains.

Processes that frequently create short-lived threads are another potential hidden source of cross-domain traffic. Since each new thread belongs to the same process and virtual memory space, it's common to reuse existing code and data structures across threads that do not overlap in time.

How to Identify Thread Migration

Tools	perf
Commands	This is an example. Adjust as necessary. While the workload is running, measure thread migrations for 30 seconds: <pre>% sudo perf record -a -e sched:sched_migrate_task -- sleep 30 % sudo perf script > raw_migr.txt % grep "comm=<insert process name of interest here>" migr.txt > my_migr.txt</pre> Post process this file line by line to extract original and destination CPU numbers for each migration. Use the CPU numbering on the system to determine which migration is cross L3 domain and intra L3 domain. [See System Checks on <i>*L3 Domains*</i>] An example output could look like: <pre>origin=7 destin=103 Intra L3 domain origin=103 destin=7 Intra L3 domain origin=100 destin=4 Intra L3 domain origin=4 destin=16 Cross L3 domain origin=18 destin=22 Intra L3 domain</pre> Cross L3 domain migrations = count of lines containing "Cross L3" Intra L3 domain migrations = count of lines containing "Intra L3" Note, the same approach can be used to measure Cross-NUMA or Cross-Socket migrations as well.
Key Indicators and Thresholds	Cross L3 Domain migrations / useful work Intra L3 Domain migrations / useful work No thresholds, meant for A vs B compares

There is no universal threshold for how many thread migrations are "too many", as it depends on factors like how much code and data is reused from the prior CPU, and the quantum of "useful work" from the standpoint of the application. As a more general approach, when testing solution A vs B, check whether *Cross L3 Domain Migrations* correlate with measured performance and cross-domain sharing.

Possible Solutions

- Consider using contemporary versions of the kernel since they may make better scheduling choices as the kernel scheduler is constantly evolving for higher performance. [See: System Checks for *Kernel Version*]
- Try to limit how often threads get rescheduled across L3 domains, since thread migration within an L3 domain of NUMA node or socket is faster than across domains. This can be achieved by setting process or thread affinities. [See: *Checking and Setting CPU Affinity*].

Further Investigation

- Try to ensure that threads from the same process are co-scheduled to the same L3 domain / NUMA node / socket. This allows any intra-process reuse of data to be of lower latency.

- Many of the techniques described for optimizing *Inter-Thread Synchronization* via more controlled scheduling may apply here.

6.1.5 Code Sharing

The indicators and motivators listed at the beginning of this section apply (*Optimizing L3 Domain Usage*). In addition, the following indicators help identify this specialized case of L3 domain interactions.

High-Level Indicators

- High % frontend_bound
- High CPU Cross-L3 Domain Code Sharing %

The code segment of a process is shared across its threads. In addition, operating systems can detect if multiple identical processes are running, and have them share code pages as well. Depending on how the threads are scheduled to underlying CPUs and L3 domains, inter-domain interactions might arise from code sharing. However, code pages are typically read-only and moreover, the footprint of most application is such that code usually fits within the caches of an L3 domain. Hence, cross-domain performance inefficiencies are generally not expected due to this. However, if the above pair of high-level indicators are detected, it is worth assessing possible solutions.

Possible Solution

- Techniques to optimize code footprint and maximize intra-domain sharing are discussed in *CPU Pipeline-Based Opportunities*

6.1.6 CPU Scheduling to L3 Domains: Spreading vs. Packing

The indicators and motivators listed at the beginning of this section apply (*Optimizing L3 Domain Usage*). In addition, the following indicators help identify this more specialized case of L3 domain interactions.

High-level Indicators

- Low CPU Usage %
- Low Per-CPU Utilization %
- High CPU Frequency Mhz

If Low CPU Usage % is detected, there is not enough work for the processor as a whole and it is under-utilized. If this is legitimate and none of the related techniques described earlier (*Common Application-Aware Tuning Opportunities*) apply, the application and operating system have many options on how and where to schedule threads. In particular, the way the threads get distributed across L3 domains can impact performance, and optimal scheduling choices can be highly application dependent.

One scheduling outcome in the Low CPU Usage % scenario is that software threads get spread across multiple L3 domains. For example, 16 threads can be spread across eight L3 domains (example: 2 logical CPUs per domain). The benefit of spread-out scheduling is that each thread has a bigger portion of shared domain resources like L3 cache and L3 domain bandwidth to the Infinity Fabric.

However, this can be a temporary or artificial benefit, as the resources may become contended once the system is fully utilized.

At the other extreme, the threads can be all packed into the same L3 domain. The benefit of packing threads from the same application into a single L3 domain is the ability to share data at high bandwidth and low latency within the L3 cache domain. This data sharing eliminates main memory traffic, improves synchronization between threads, and reduces producer/consumer thread communication latency. These benefits show up in both Low CPU Usage % and High CPU Usage % scenarios.

Either outcome can be beneficial depending on the application, while an in-between outcome of non-uniform spreading/packing is likely to produce poor performance behaviors.

Possible Solutions

- If the default scheduling choice does not match expectations and/or is impacting performance negatively, the behavior can be modified. Use techniques for setting CPU affinities (*CPU Model and Number*) to optimize this.

CPU PIPELINE-BASED OPPORTUNITIES

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

This section will cover optimization opportunities gleaned from microarchitectural indicators, which may lead to general or EPYC-specific opportunities.

APPLICATION SCALING OPPORTUNITIES

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

High-Level Indicators

- Low Per-CPU Utilization % OR Moderate Per-CPU Utilization %
- High CPU Usage %

Possible Motivator

Unexpected Application Scaling

A common objective of performance optimization is to ensure that applications scale to take advantage of the degree of parallelism and throughput capabilities offered by the high core counts of AMD EPYC™ Series Processors. This section describes how to detect factors that may limit such performance scalability, and possible solutions for optimization. This will often point to scenarios that have already been discussed elsewhere in this document.

8.1 Types of Performance Scaling

There are two ways applications take advantage of the high logical CPU counts offered by modern systems.

Scale-Up Performance: This is the performance response obtained by giving an application increased degree of parallelism of underlying compute resources by increasing the number of available logical CPUs. This scaling methodology is often associated with highly performance-tuned applications, commonly encountered in enterprise or HPC deployments, where it's referred to as "Strong Scaling".

Scale-Out Performance: This is the performance response obtained by scaling the number of instances of an application (or the amount of work with compute). In such a deployment, instances work relatively independently of each other, each limited to a fixed number of logical CPUs (using containers or a Virtual Machine). This scaling methodology is commonly seen in cloud environments and HPC environments. In the latter, it's referred to as "Weak Scaling", where MPI ranks are analogous to workload instances.

8.2 Scaling Efficiency

Regardless of which methodology is being used, *Scaling Efficiency* is an important performance indicator in scaling context. This section defines this indicator, provides instructions on how to measure it, and then discusses the next-level indicators that influence it (some of which have already been discussed elsewhere in this Playbook preview), and therefore provides pointers to relevant part of this document for optimization.

In a scale-out context, overall performance is the product of performance per application instance and the number of instances. In a scale-up context, overall performance is simply the performance seen by the single (and only) instance of the application.

The rest of this section assumes application-level considerations have been used to determine the performance metric under scaling and assumes a hypothetical throughput-oriented application as an example.

How to calculate Scaling Efficiency when scaling the number of CPUs from X to Y (increased CPU Usage %)

Tools	End-application performance measurement
Commands, Indicators, and Thresholds	Scaling Efficiency % = ((Throughput at Y CPUs / Throughput at X CPUs) / (Y / X) - 1) x 100 Hence, a 2X improvement in throughput with a 2X increase in CPUs represents a 100% scaling efficiency. If no performance was gained by doubling the CPU count, that represents a 50% scaling efficiency. Which implies that removing half the CPUs would not hurt performance. • High Scaling Efficiency %: > 90% • Moderate Scaling Efficiency %: 80%-90% • Low Scaling Efficiency %: 50%-80% • Negative Scaling Efficiency %: < 50%

If Low Scaling Efficiency is detected at a target examine the following high level indicators (see: [Performance Indicators](#)) at those CPU counts, considering only the active logical CPUs:

- Per-CPU Utilization %
- IPC w/o halt

Possible Solution

- If Per-CPU Utilization % falls off with increasing CPU Usage %, investigate techniques for `opt_cpu_usage` and [Inter-Thread Synchronization](#).
- If IPC w/o halt decreases at High CPU Usage %, it needs further analysis. This could be due to multiple factors. Examine the [CPU Performance Indicators](#) view at any two chosen points of interest and determine which indicator has worsened. Often, this turns out to be % `backend_bound_memory`. Common indicators that accompany worsening of IPC w/o halt with CPU Usage % are provided below, along with potential solutions. Examine which indicator got worse, and take appropriate action:

Worsening High Level Indicator	Optimizing IPC w/o halt at High CPU Usage %
High CPU Cross-L3 Domain Data Sharing %	Scaling the number of CPUs beyond the number of logical CPUs within an L3 domain can drive increases in inter-L3 domain interactions. See Optimizing L3 Domain Usage .

CAPTURING INDICATORS USING AMD UPROFPCM

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

The following table lists the various performance indicators that have been referred to in the document and the AMD uProfPcm commands that produce them. It also provides a way to map the indicator name, as referred to in this document, to the raw metric names (surrounded by []) generated by uProfPcm.

Note, these individual uProfPcm commands can be combined into fewer commands to ease collection of these indicators. For example, the following single command will produce all the indicators listed in the table below.

Finally, the commands below profile all the logical CPUs in the system. When using these, care should be exercised to profile only the CPUs on which the application is running. See [AMD uProf](#) for further details.

```
% AMDuProfPcm -m ipc,pipeline_util,l1,tlb,dc,l2,l3,cache_miss, -O <output_dir> -A_
↪system -a -d <duration>
```

Indicator	Command
Per-CPU Utilization %	AMDuProfPcm -m ipc -O <output_dir> -A system -a -d <duration> [Utilization (%)]
CPU Frequency Mhz	AMDuProfPcm -m ipc -O <output_dir> -A system -a -d <duration> [Eff Freq (MHz)]
% frontend_bound	AMDuProfPcm -m pipeline_util -O <output_dir> -a -d <duration> [Frontend_Bound]
% backend_bound_memory	AMDuProfPcm -m pipeline_util -O <output_dir> -a -d <duration> [Backend_Bound.Memory]
% backend_bound_cpu	AMDuProfPcm -m pipeline_util -O <output_dir> -a -d <duration> [Backend_Bound.CPU]
% smt_contention	AMDuProfPcm -m pipeline_util -O <output_dir> -a -d <duration> [SMT_Disp_contention]
Instruction Cache Miss	AMDuProfPcm -m l1 -O <output_dir> -A system -a -d <duration> [L1 IC Miss (pti)]
Branch Mispredicts	AMDuProfPcm -m ipc -O <output_dir> -A system -a -d <duration> [Retired Branches Mispredicted (pti)]
Data Cache Miss	AMDuProfPcm -m cache_miss -O <output_dir> -A system -a -d <duration> [L1 DC Miss (pti)]
L2 ITLB Miss	AMDuProfPcm -m tlb -O <output_dir> -A system -a -d <duration> [L2 ITLB Miss (pti)]
L2 DTLB Miss	AMDuProfPcm -m tlb -O <output_dir> -A system -a -d <duration> [L2 DTLB Miss (pti)]
L2 Cache Miss	AMDuProfPcm -m l2 -O <output_dir> -A system -a -d <duration> [L2 Miss (pti)]
L3 Cache Miss	AMDuProfPcm -m l3 -O <output_dir> -A system -a -d <duration> [L3 Miss (pti)] (Planned in uProf 5.1)
CPU Cross L3 Domain Data Sharing %	AMDuProfPcm -m dc -O <output_dir> -A system -a -d <duration> [DC Fills From Remote CCX Cache (pti) + DC Fills From different CCX in same node (pti) / All DC Fills (pti)]

PROCESSOR-SPECIFIC SOLUTIONS

Note: This is a *preview* of the AMD EPYC Processor Performance Playbook. It illustrates the Playbook methodology by focusing on a subset of the optimization space, specifically, performance optimization opportunities in the presence of multiple L3 cache domains.

10.1 Adaptive Allocation

Scenarios where manipulating this feature may deliver performance are discussed under techniques to optimize L3 domain usage [See *Producer and Consumer*].

10.1.1 3rd-Gen EPYC (Milan, Milan-X)

Options to manipulate Adaptive Allocation are available under NDA.

10.1.2 4th-Gen EPYC (Genoa, Genoa-X, Bergamo, Siena)

Options to manipulate Adaptive Allocation are available under NDA.

10.1.3 5th-Gen EPYC (Turin)

Starting with Turin, BIOS options exist to enable/disable Adaptive Allocation.

Tools	BIOS
Commands	CPU Common Options → Adaptive Allocation (AA) → [Auto/Enabled/Disabled]

10.2 Speculative Stores Mode Configuration

The trade-offs related to this feature are discussed in *Optimizing L3 Domain Usage*. Generation-specific guidance for how to manipulate this feature is provided below.

10.2.1 3rd-Gen EPYC (Milan, Milan-X)

This feature is not available on 3rd-Gen EPYC.

10.2.2 4th-Gen EPYC (Genoa, Genoa-X, Bergamo, Siena)

The 4th-Gen default is ‘Balanced’. Setting to ‘More Speculative’ will produce behaviors that are similar to 3rd-Gen EPYC.

Configure Speculative Store Mode

Tools	BIOS
Commands	CPU Speculative Store Modes {Balance, More Speculative, Less Speculative} Balanced: Store instructions may delay sending out their invalidations to remote cacheline copies when the cacheline is present but not in a writable state in the local cache. More Speculative: Store instructions will send out invalidations to remote cacheline copies as soon as possible. Less Speculative: Store instructions may delay sending out their invalidations to remote cacheline copies when the cacheline is not present in the local cache or not in a writable state in the local cache.

10.2.3 5th-Gen EPYC (Turin)

5th-Gen adds an “Aggressive” option in addition to the ones listed above [FIXME].

EXTERNAL REFERENCES

1. The Art of Multiprocessor Programming, Herlihy and Shavit, Morgan Kauffman, 2nd Edition, 2020.
2. Lam, Monica D., Edward E. Rothberg, and Michael E. Wolf. “The cache performance and optimizations of blocked algorithms.” ACM SIGOPS Operating Systems Review 25, no. Special Issue (1991): 63-74.
3. Optimizing Compilers for Modern Architectures: A Dependence-Based Approach, Ken Kennedy, John R. Allen Morgan Kaufmann Publishers Inc. 2001.
4. *Ding-Kai Chen, Hong-Men Su, Pen-Chung Yew*, “The impact of synchronization and granularity on parallel systems”, ACM SIGARCH Computer Architecture News, Volume 18, Issue 2SI, 1990.
5. *Vladimir Vlassov, Oscar Sierra Merino, Csaba Andras Moritz, Konstantin Popov*, “Support for fine- grained synchronization in shared-memory multiprocessors”, *PaCT’07: Proceedings of the 9th* *international conference on Parallel Computing Technologies, Sep. 2007*.

ADDITIONAL RESOURCES AND LEGAL NOTICES

12.1 Revision History

Table I.1: Revision History

Date	Revision	Description
2024-12-04	1.0	First version of this document.

12.2 Notices

© Copyright 2024 Advanced Micro Devices, Inc.

The information presented in this document is for informational purposes only and may contain technical inaccuracies, omissions, and typographical errors. The information contained herein is subject to change and may be rendered inaccurate for many reasons, including but not limited to product and roadmap changes, component and motherboard version changes, new model and/or product releases, product differences between differing manufacturers, software changes, BIOS flashes, firmware upgrades, or

the like. Any computer system has risks of security vulnerabilities that cannot be completely prevented or mitigated. AMD assumes no obligation to update or otherwise correct or revise this information.

However, AMD reserves the right to revise this information and to make changes from time to time to the content hereof without obligation of AMD to notify any person of such revisions or changes.

THIS INFORMATION IS PROVIDED “AS IS.” AMD MAKES NO REPRESENTATIONS OR WARRANTIES WITH RESPECT TO THE CONTENTS HEREOF AND ASSUMES NO RESPONSIBILITY FOR

ANY INACCURACIES, ERRORS, OR OMISSIONS THAT MAY APPEAR IN THIS INFORMATION. AMD SPECIFICALLY DISCLAIMS ANY IMPLIED WARRANTIES OF NON-INFRINGEMENT, MERCHANTABILITY, OR FITNESS FOR ANY PARTICULAR PURPOSE. IN NO EVENT WILL AMD BE LIABLE TO ANY PERSON FOR ANY RELIANCE, DIRECT, INDIRECT, SPECIAL, OR OTHER CONSEQUENTIAL DAMAGES ARISING FROM THE USE OF ANY INFORMATION CONTAINED HEREIN, EVEN IF AMD IS EXPRESSLY ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

12.3 Trademarks

AMD, the AMD Arrow logo, and combinations thereof are trademarks of Advanced Micro Devices, Inc.

Other product names used in this publication are for identification purposes only and may be trademarks of their respective companies.